

malinha

Planejador de viagens

Guia do código e funcionamento

Da interface ao banco de dados, passando pelas APIs, pelo Gemini e pela navegação entre os dias do roteiro.

Material de estudo e apoio à apresentação acadêmica

Explicações por funcionalidade, trechos reais do código e um mapa das classes Java.

Versão do projeto consultada em 11 de novembro de 2026.

SUMÁRIO

Como usar este guia

Leia os capítulos 1 a 8 para entender o núcleo do aplicativo. Os capítulos seguintes explicam cada integração. Os apêndices permitem localizar os arquivos e os pontos de manutenção.

O que o Malinha faz 3

Como o projeto se organiza 4

Da abertura à navegação 5

Planejar viagem, passo a passo 6

Os objetos que representam os dados 7

Adicionar, editar, excluir e ordenar 8

Como funciona o deslize entre dias 9

Persistência de viagens em SQLite 10

Descobrir destinos e usar modelos 11

Fotos reais, cache e atribuição 12

Autocomplete de cidades e lugares 13

Mapa, atrações e hospedagem 14

Câmbio e cálculo com BigDecimal 15

Como o Gemini vira atividades 16

Aeroportos e busca de voos 17

Links dos comparadores e reservas 18

Países, feriados e calendário 19

Recursos XML, build e configurações 20

Tratamento de estado e erros 21

Como testar e explicar o projeto 22

Mapa dos arquivos Java 23

Mapa dos arquivos Java 24

Mapa dos arquivos Java 25

Mapa dos arquivos Java 26

Mapa dos arquivos Java 27

Onde alterar cada recurso 28

Glossário e fontes 29

01 / VISÃO GERAL

O que o Malinha faz

O Malinha ajuda o usuário a descobrir destinos, montar uma viagem, organizar atividades por dia e recuperar o roteiro depois. O projeto é um aplicativo Android acadêmico, com interface em português do Brasil e código de aplicação em Java.

1. Explorar ou escolher destino
2. Preencher o planejamento
3. Organizar os dias
4. Salvar e reabrir

Três formas de iniciar

- Montagem manual pelo assistente Planejar viagem.
- Modelo local de roteiro pronto, com atividades editáveis.
- Sugestões do Gemini adicionadas pelo usuário ao roteiro, na variante debug.

O que funciona sem internet

O catálogo de destinos, os modelos de roteiro, o editor, o banco de viagens, a pesquisa local de aeroportos e as reservas anotadas usam dados do aparelho. Consultas de fotos, lugares, câmbio, países, feriados e IA dependem da rede e da configuração do serviço.

> Este guia explica o código encontrado em 11/11/2026. As verificações automatizadas citadas adiante são registros da implementação anterior. A preparação deste PDF não executa o app nem novas chamadas às APIs.

02 / ARQUITETURA

Como o projeto se organiza

A arquitetura é centrada em Activities. Cada tela conecta o XML aos eventos do usuário. Adaptadores apresentam coleções, serviços cuidam das integrações e classes utilitárias concentram tarefas compartilhadas. O fluxo principal mantém estado nas Activities e em Bundle.

Camada	Responsabilidade
activities	Telas, navegação, formulários, diálogos e coordenação das ações.
models	Representação dos dados de viagem, atividade, destino, cidade e país.
adapters	Ligação entre coleções Java e cartões ou páginas no RecyclerView.
database	Criação, migração, leitura, gravação e exclusão no SQLite.
services	Consultas HTTP, clientes Google e construção de links.
utils	Datas, validação, catálogos, autocomplete e vínculo das fotografias.
res e assets	Layouts, textos, cores, drawables e a base offline de aeroportos.

Como ler uma tela

Comece pelo onCreate(): ele define o layout, localiza views pelo ID, restaura os dados e registra os listeners. Siga o método chamado pelo botão principal. Depois confira o serviço ou banco acionado e o adaptador que atualiza a lista.

> O Gradle usa arquivos build.gradle.kts. Essa escolha configura o build e não muda a linguagem Java das telas.

03 / ENTRADA E CICLO DE VIDA

Da abertura à navegação

Arquivo	Papel
AndroidManifest.xml	Declara permissões, Activities, tema, metadados e a entrada da aplicação.
SplashActivity.java	Mostra a abertura e encaminha ao início. Gerencia o callback de navegação.
HomeActivity.java	Liga os seis atalhos, a navegação inferior e os cartões de próximas viagens.
BaseActivity.java	Compartilha Toolbar, menu de opções e tratamento das áreas do sistema.

Áreas seguras e teclado

BaseActivity.applyInsets() usa WindowInsetsCompat para considerar as barras do sistema, recortes de tela e teclado. O padding inicial é preservado e somado aos insets, evitando que os controles fiquem encobertos.

Menu de opções

onCreateOptionsMenu() infla app_options.xml. onOptionsItemSelected() identifica a opção escolhida e mostra ajuda ou informações sobre o app. Esses métodos também demonstram o requisito acadêmico de menu.

Por que existe Bundle

O Android pode recriar a Activity ao girar a tela. onSaveInstanceState() registra os dados temporários necessários. Ao reabrir a tela, onCreate() e, quando usado, onRestoreInstanceState() recuperam esses valores. Bundle preserva uma sessão de interface. Salvar roteiro grava no SQLite para recuperação posterior.

MainActivity, FirstFragment e SecondFragment permanecem do template inicial. O launcher registrado é SplashActivity.

04 / FORMULÁRIO GUIADO

Planejar viagem, passo a passo

PlanTripActivity.java usa activity_plan_trip.xml. Os três primeiros passos alternam a visibilidade de painéis na mesma Activity. O quarto abre ItineraryDraftActivity com o rascunho.

Passo	Código e comportamento
1. Destino	Campo editável, autocomplete de cidades e validação de destino obrigatório.
2. Datas	DatePickerDialog, atalhos, validação do intervalo e cálculo de noites.
3. Viajantes e estilo	Stepper, ritmo, orçamento decimal e seleção dos interesses.
4. Roteiro	Editor já existente, recebido por ActivityResultLauncher.

Avançar e voltar

advance() valida o passo atual e decide se mostra o próximo painel ou chama createDraft(). showStep() atualiza o título, o progresso e os botões. goBack() retorna um painel e fecha a tela quando o usuário já está no primeiro.

Criação do rascunho

createDraft() valida destino, datas, orçamento e interesses. Preenche uma Viagem, informa quantidade de viajantes e ritmo e envia o objeto por Intent. Na edição de uma viagem salva, também verifica se o novo período comporta todas as atividades.

Atalhos de data

Fim de semana e daqui a 30 dias usam Calendar localmente. Próximo feriado consulta Nager.Holidays para o Brasil em uma thread de trabalho. A resposta só é aplicada se a consulta ainda for válida e a Activity continuar ativa.

> O orçamento aceita formatos monetários tratados por PlanningUtils.parseBudget(). Entradas inválidas impedem o avanço.

05 / MODELS

Os objetos que representam os dados

Viagem e Atividade implementam Serializable, o que permite transportá-las entre Activities e preservá-las no estado da tela. A gravação permanente é feita separadamente por TripDatabase.

```
public long id = -1, start, end;
    public int travelers = 1, pace = 1;
    public String destination = "", budget = "0";
    public ArrayList<String> interests = new ArrayList<>();
    public ArrayList<Atividade> activities = new ArrayList<>();
```

Como interpretar Viagem

id = -1 identifica uma viagem ainda não inserida. start e end são instantes em milissegundos. travelers e pace registram preferências. budget é texto decimal. interests e activities são ArrayList. A ordem da lista de atividades é a ordem escolhida no roteiro.

```
public String id = UUID.randomUUID().toString();
    public int day, startMinutes, endMinutes, category;
    public String name = "", description = "", cost = "0";
```

Como interpretar Atividade

O UUID mantém a identidade de uma atividade durante edição e reordenação. day começa em zero. startMinutes e endMinutes contam minutos desde a meia-noite: 09h30 corresponde a 570. category aponta para a lista de categorias dos recursos. cost é armazenado como texto decimal.

> Outros models incluem Destino, com IDs de recursos e região, SelectedCity, com identidade e coordenadas, e CountryTravelData, com dados do país e feriados.

06 / EDITOR DO ROTEIRO

Adicionar, editar, excluir e ordenar

ItineraryDraftActivity.java coordena a lista completa de atividades. Ela recebe uma Viagem pelo Intent ou consulta uma viagem pelo trip_id. Ao recriar a tela, dá prioridade ao estado salvo da sessão.

1. Tocar em Adicionar
2. Abrir EditActivityActivity
3. Validar e confirmar
4. Receber o resultado
5. Atualizar a página do dia

Edição e confirmação

EditActivityActivity usa TimePickerDialog e campos XML para local, descrição, custo e categoria. confirm() exige nome, horário final posterior ao inicial e custo válido. setResult(RESULT_OK, Intent) devolve o objeto. A Activity do roteiro substitui a atividade com o mesmo UUID ou acrescenta uma nova.

Exclusão e reordenação

ActivityAdapter expõe ações de editar, excluir, mover e abrir mapa. A exclusão pede confirmação. ItineraryUtils.move() altera a sequência apenas entre atividades do mesmo dia. A ordenação usa os botões Subir e Descer, evitando conflito entre gesto horizontal e troca de dia.

Alterações pendentes

dirty indica que há mudanças a salvar. saveTrip() escreve o conjunto no banco e atualiza o ID. requestExit() consulta esse estado para pedir confirmação ao descartar alterações.

> Horários devem terminar no mesmo dia. A ordem é manual e pode diferir da ordem cronológica. Atividades simultâneas são permitidas pelo editor.

07 / VIEWPAGER2

Como funciona o deslize entre dias

Componente	Responsabilidade
ViewPager2	Página horizontal com largura da tela e suporte nativo a navegação acessível.
TabLayout e TabLayoutMediator	Abas roláveis, indicador e sincronização com o pager.
ItineraryDayAdapter	Cria uma lista vertical para cada dia e preserva sua posição.
ActivityAdapter	Apresenta e opera as atividades de uma página.
itinerary_day_count	Exibe o contador, como 2 de 7, e anuncia mudanças de forma acessível.

Composição de uma página

Cada página contém um RecyclerView vertical. Um ConcatAdapter combina o cabeçalho do destino, o ActivityAdapter e a mensagem de dia vazio. Esse arranjo permite que foto e informações também participem da rolagem. Dias vazios oferecem um botão de adição.

Escolha do dia inicial

todayIndex() compara ano e dia do ano das datas da viagem com o calendário atual. Se encontrar hoje, abre essa página. Fora do período, abre o Dia 1. Quando existe Bundle, o dia restaurado prevalece e é ajustado aos limites da viagem.

Restauração e listas independentes

selectedDay é atualizado por OnPageChangeCallback. ItineraryDayAdapter guarda o estado de cada LinearLayoutManager em SparseArray<Parcelable>, associado ao índice do dia. Ao reciclar ou recriar a página, o estado correspondente é restaurado.

> O pager usa ViewPager2 1.1.0 e RecyclerView 1.3.2. Tocar em uma aba também anima a mudança de página.

08 / BANCO DE DADOS

Persistência de viagens em SQLite

TripDatabase estende SQLiteOpenHelper e usa malinha.db. A versão atual do esquema é 2. A tabela trips guarda uma viagem completa por linha.

Colunas	Representação
id	Chave primária numérica com autoincremento.
destination, start, end	Destino e limites do período.
budget, interests, activities	Orçamento em texto e listas serializadas como JSON.
travelers, pace	Quantidade de viajantes e ritmo, acrescentados na versão 2.

Salvar o conjunto

save() transforma as atividades em JSONArray, prepara ContentValues e faz uma inserção para ID negativo ou atualização para uma viagem existente. A gravação em uma única linha mantém destino, preferências e atividades no mesmo conjunto persistido. Uma atualização que não encontra a viagem gera erro.

Ler e excluir

list() consulta viagens ordenadas pelo início. find(id) recupera uma linha. read(Cursor) reconstrói Viagem e Atividade a partir das colunas e do JSON. delete(id) remove a viagem selecionada. Cursors e conexões são fechados pelos blocos try-with-resources.

Migração compatível

onUpgrade() acrescenta travelers e pace com valores padrão quando a versão anterior é menor que 2. As linhas existentes continuam com os demais campos e listas preservados.

Salvar na tela atualiza o SQLite. Bundle e ArrayList mantêm o trabalho temporário até essa confirmação.

09 / EXPLORAR

Descobrir destinos e usar modelos

ExploreActivity.java usa o catálogo local de DestinationCatalog. O conjunto contém 24 destinos, com IDs de nome, descrição, imagem e região. O filtro de texto normaliza acentos e caixa, e os chips combinam categoria com a seleção regional.

Carrosséis e favoritos

As seções apresentam cartões horizontais por um DestinationAdapter. Os favoritos usam SharedPreferences locais com chaves baseadas nos nomes dos recursos. CommonsPhotoView solicita as imagens e mantém os créditos associados.

Detalhes e planejamento

details() monta um diálogo rolável com resumo, orientação sobre quando ir, atrações e atalhos. O botão Usar como base do roteiro chama PlanTripActivity com prefill_destination. Países e feriados recebe o código do país. A pesquisa de atrações recebe o destino.

Roteiros prontos

ReadyItineraries cria novas atividades a cada escolha. Há modelos de cinco dias para Rio de Janeiro, Salvador, Santiago, Buenos Aires, Lisboa e Nova York, seis para Paris e sete para Tóquio. Cada dia tem quatro paradas sugeridas.

previewReady() mostra os dias e horários. chooseReadyStart() pede uma data, define o período com Calendar e abre uma nova Viagem no editor. O usuário pode alterar o conteúdo e precisa salvar a cópia.

As categorias e modelos são curadoria local. Horários, ingressos e disponibilidade dos locais precisam de conferência pelo viajante.

10 / WIKIMEDIA E GLIDE

Fotos reais, cache e atribuição

1. Nome do local e destino
2. Wikipédia em português / PageImages
3. Pesquisa no Commons
4. Metadados da foto
5. Glide e crédito no cartão

WikimediaImageService é um serviço reutilizável com uma instância compartilhada. find() recebe assunto, contexto e callback. A consulta acontece em ExecutorService e o resultado volta para a thread principal por Handler.

Seleção de imagens

lookup() tenta localizar o arquivo associado ao artigo da Wikipédia em português. Se necessário, resolve nomes equivalentes e pesquisa no Commons com local, cidade e país. O serviço examina os metadados e mantém autoria, licença e origem na resposta Photo.

Cache e consultas simultâneas

O serviço usa LruCache para resultados e equivalências de nomes. HIT_TTL vale 24 horas e MISS_TTL vale 15 minutos. O mapa pending reúne callbacks de consultas idênticas em andamento para reaproveitar o resultado.

Por que a foto não troca de cartão

CommonsPhotoView atribui uma marca ao ImageView antes da consulta. O callback só aplica a resposta se a marca continuar igual. Isso evita que a resposta de um cartão reciclado apareça em outro. Glide mantém o cache de download e mostra o drawable padrão durante a carga ou falha.

> Os créditos acompanham o arquivo. Autor, licença e página de origem são dados da fonte e precisam continuar acessíveis.

11 / GOOGLE PLACES

Autocomplete de cidades e lugares

Classe	Papel
PlacesClientProvider	Inicialização compartilhada do Places com a API New e cliente configurado.
CitySearchRepository	Previsões de cidades e obtenção dos detalhes da seleção.
CityAutocompleteController	TextWatcher, debounce, visibilidade das sugestões e ciclo de vida.
CitySuggestionAdapter	Texto principal e secundário das sugestões.
ExplorePlacesService / HotelPlacesService	Pesquisas de atrações, restaurantes e hospedagens.

Evitar uma chamada por tecla

O controlador espera cerca de 300 ms após a última mudança e consulta a partir de três caracteres. removeCallbacks() cancela buscas ainda agendadas. Um contador de geração impede que respostas antigas alterem a consulta atual.

Sessão de pesquisa

CitySearchRepository mantém AutocompleteSessionToken como campo. Usa PlaceTypes.CITIES na busca. Na seleção, envia o mesmo token para fetchPlace e solicita somente ID, DISPLAY_NAME e LOCATION. O resultado vira SelectedCity. A sessão é renovada após a seleção.

Selecionar sem reiniciar a busca

O controlador suspende temporariamente o TextWatcher enquanto preenche o campo com a cidade escolhida. No erro, mostra uma mensagem e registra uma identificação limitada da falha. Ao destruir a tela, remove callbacks e libera a ligação com a lista.

> Esta integração exige configuração Places e rede. O autocomplete de aeroportos da tela Voos usa outro mecanismo, inteiramente local.

12 / LOCALIZAÇÃO E SERVIÇOS EXTERNOS

Mapa, atrações e hospedagem

MapActivity recebe a consulta do local da atividade junto com o destino. O mapa usa SupportMapFragment e Maps SDK for Android. A resolução de nome para endereço usa o Geocoder do Android em segundo plano.

Escolha do endereço

A busca pode retornar locais ambíguos. A tela permite escolher um endereço antes de marcar o ponto. O mapa mostra a localização escolhida, com zoom e navegação próprios do SDK. Abrir a rota encaminha para o Google Maps pelo navegador ou aplicativo disponível.

Dados de atrações e restaurantes

ExplorePlacesActivity coordena categoria, consulta, lista de resultados e paginação. ExplorePlaceAdapter apresenta os dados recebidos e as ações disponíveis. A consulta de horários é feita sob demanda, com descarte de respostas de uma seleção anterior.

Hotéis

HotelsActivity continua no fluxo compartilhado TravelSearchActivity. O formulário valida destino, período e quantidade de hóspedes. HotelResultsActivity usa HotelPlacesService para listar estabelecimentos. A pesquisa externa de hospedagem é montada por TravelSearchService.hotels().

Abrir links

TravelSearchService usa Uri e Intent ACTION_VIEW para encaminhar a pesquisa. Os parâmetros são codificados. ActivityNotFoundException e SecurityException são tratados com mensagem ao usuário.

Notas, avaliações, horários e localização refletem os dados disponíveis no serviço. O app depende do fornecedor para esses resultados.

13 / AWESOMEAPI

Câmbio e cálculo com BigDecimal

CurrencyActivity recebe moedas e valor e chama CurrencyService fora da thread principal. O endpoint utilizado é economia.awesomeapi.com.br/json/last/{pares}. A implementação atual lê o campo bid, o código de moeda e o timestamp.

Conversões por BRL

Se a origem for BRL, o serviço obtém a cotação da moeda de destino em reais e inverte a relação. Se o destino for BRL, utiliza a cotação da origem. Quando as duas moedas são estrangeiras, divide o valor da origem em BRL pelo valor do destino em BRL.

Exemplo conceitual	Resultado
Origem em BRL e destino em USD	Taxa calculada como 1 dividido pela cotação USD-BRL.
Origem em USD e destino em BRL	Taxa baseada em USD-BRL.
Origem em EUR e destino em USD	Relação entre EUR-BRL e USD-BRL.
Moedas iguais	Taxa 1 calculada no aparelho.

Precisão e validação

Valores são tratados com BigDecimal. CurrencyService usa MathContext.DECIMAL128 na divisão. O arredondamento de apresentação respeita a moeda. Respostas com pares inesperados, valores não positivos ou timestamp inválido geram erro. A conexão tem timeout e fechamento no finally.

> Frankfurter aparece nos documentos das primeiras etapas. A classe atual de câmbio consulta AwesomeAPI. Uma cotação de referência não inclui automaticamente custos de uma operação financeira.

14 / FIREBASE AI LOGIC

Como o Gemini vira atividades

1. Usuário pede sugestões
2. App Check
3. Prompt com contexto e exclusões
4. JSON do Gemini
5. Seleção de cartões
6. Atividades do roteiro

FirebaseAIService fica no pacote com.example.malinha. O modelo configurado no código inspecionado é gemini-3.1-flash-lite. suggest() monta GenerationConfig com application/json e Schema, obtém GenerativeModelFutures e retorna ListenableFuture.

Campo do JSON	Finalidade
name, city, country	Identificar a atração e o contexto.
category, description	Apresentar o tipo e um resumo.
duration_minutes, suggested_time	Sugerir a ocupação do dia.
cost	Informação aproximada quando fornecida.

Seleção e adição

AiSuggestionsSheet transforma o JSON em objetos e mostra os cartões no BottomSheetDialog. Pedir mais opções inclui as atrações já sugeridas para reduzir repetições e preserva as seleções. As fotos vêm da Wikimedia.

ItineraryDraftActivity.addSuggestions() prepara um lote, compara nomes com atividades existentes e procura intervalos entre 09h e 22h. Só acrescenta o lote quando consegue validar as opções. Depois disso, as atividades seguem o fluxo normal de edição e salvamento.

Erros e autorização

failureMessage() distingue problemas de rede, App Check, quota, modelo e permissão. DebugMalinhaApplication instala o provedor de teste antes das chamadas. O botão de IA está limitado ao build debug no código atual.

> O token de teste precisa ser cadastrado para cada ambiente. A disponibilidade do modelo e as cotas dependem do projeto Firebase configurado.

15 / AUTOCOMPLETE OFFLINE

Aeroportos e busca de voos

FlightsActivity usa activity_flights.xml e possui seu próprio formulário. Origem e destino são AutoCompleteTextView ligados a AirportAutocomplete. O arquivo assets/airports.tsv contém 6.071 registros com IATA, nome, cidade, país, latitude, longitude e fuso.

Como as sugestões são encontradas

A primeira pesquisa carrega o arquivo em memória. O Filter do adaptador executa a leitura e a filtragem fora da thread de interface. A partir de dois caracteres, a busca normaliza acentos e caixa, compara código, cidade e nome e ordena os melhores resultados. São exibidas até oito sugestões por vez.

O ranking favorece correspondência exata, cidade e prefixo. Entre resultados equivalentes, considera aeroportos brasileiros e hubs. Alguns nomes comuns recebem equivalentes em português, como Lisboa, Londres, Nova York e Tóquio.

Enviar o código ao comparador

O rótulo tem o formato Cidade (IATA) · País. searchValue() extrai o IATA da sugestão selecionada. A inversão, a restauração da última origem e o histórico usam setText(..., false), evitando disparar uma nova seleção automática.

Histórico

A pesquisa válida é representada em JSONObject e guardada em SharedPreferences. O histórico mantém até cinco consultas e remove repetições idênticas. Tocar em uma pesquisa restaura rota, datas, adultos e classe.

O dataset OpenFlights está sob ODbL. O arquivo de atribuição e o botão de créditos identificam a fonte e a transformação para TSV.

16 / VOOS NO APARELHO

Links dos comparadores e reservas

Montagem dos links

FlightSearchLinks centraliza os fornecedores e suas páginas de busca. build() recebe origem, destino, ida, volta opcional, adultos e Cabin. Google Voos recebe uma consulta textual codificada. Os demais usam códigos IATA e formatos de datas próprios. A Decolar abre a página de pesquisa para preenchimento manual. FlightsActivity.compare() valida a rota e o período, gera o URI, salva a consulta recente e abre o serviço por TravelSearchService. Um fallback avisa que será necessário completar a busca no site. Os filtros efetivos são controlados pelo comparador.

Meus voos

FlightReservations abre a lista de reservas anotadas e um formulário de criação ou edição. Os campos incluem companhia, número, origem, destino, datas e horários locais, localizador e observações. A origem, o destino e a partida são obrigatórios na anotação.

Privacidade e gravação

O localizador usa transformação de senha, com opção explícita para revelar. O texto não entra nos logs. As reservas são armazenadas em flight-reservations.json dentro de getNoBackupFilesDir(). AtomicFile permite concluir uma gravação ou recuperar a versão anterior se a escrita falhar.

> As datas da reserva são anotações de horário local. O arquivo atual ainda não calcula instantes por fuso, vincula reservas ao SQLite de viagens nem agenda lembretes.

17 / CONSULTAS INDEPENDENTES

Países, feriados e calendário

CountryGuideActivity apresenta país e ano. As consultas são feitas em ExecutorService, com estado de carregamento e identificação da geração da requisição. Uma falha nos dados do país permite que os feriados recebidos pela outra consulta continuem disponíveis.

Classe	Papel
CountryHttp	HTTP compartilhado, validação de código e autenticação quando aplicável.
RestCountriesService	Dados do país pelo endpoint v5 com token Bearer.
NagerDateService	Feriados pelo endpoint v4, com validação de país e ano.
HolidayWeekends	Cálculo local dos intervalos prolongados.
CountryCalendarAdapter	Apresentação dos itens de calendário.
CountryTravelData	Estruturas para idiomas, moedas, fusos, telefones e feriados.

Dados do país

O endpoint seleciona nome, código, idiomas, moedas, fusos e códigos telefônicos. O parser verifica se o país da resposta é o solicitado e rejeita respostas marcadas como amostras de demonstração.

Fins de semana prolongados

HolidayWeekends usa feriados nacionais do tipo Public e descanso aos sábados e domingos. A combinação é calculada no aparelho para o ano consultado. Ela não representa uma confirmação sobre folgas, emendas ou regras pessoais de trabalho.

> Os endpoints configurados são nagerholidays.com/api/v4/Holidays/{país}/{ano} e api.restcountries.com/countries/v5/codes.alpha_2/{código}. A chave REST Countries é uma configuração separada das chaves Google.

18 / COMO O APK É MONTADO

Recursos XML, build e configurações

Arquivo ou diretório	O que controla
app/build.gradle.kts	Application ID, SDKs, dependências, build types e placeholders.
build.gradle.kts e gradle/	Plugins, wrapper e toolchain usados pelo projeto.
res/layout	Estrutura das telas e cartões ligados por R.id.
res/values	Textos, cores, dimensões, temas e arrays.
res/drawable	Mascote, ícones, bordas e ilustrações locais.
app/src/debug	Inicialização específica da variante de demonstração.

Versões do projeto inspecionado

O Application ID é com.example.malinha. O minSdk é 24, compileSdk e targetSdk são 37. O código Java usa compatibilidade 11 e o arquivo de toolchain solicita JDK 25. O Firebase BoM está em 35.1.0. Os números são os declarados no projeto, sem atualização de dependências para produzir este guia.

Configurações locais

maps.properties define MAPS_API_KEY. places.properties define PLACES_API_KEY. app/google-services.json conecta o app ao Firebase. O recurso rest_countries_api_key é usado para REST Countries. flights.properties e flight-proxy/.env pertencem à integração legada de preços indicativos.

As chaves de SDK Android precisam de restrições de pacote, certificado e API. Um token privado de servidor exige armazenamento e controle adequados ao seu uso. O guia apresenta nomes de configuração e não reproduz valores privados.

> As integrações do app são opcionais para o editor offline. Modificar uma propriedade embutida no build exige gerar e instalar outro APK.

19 / PADRÕES QUE SE REPETEM

Tratamento de estado e erros

Situação	Mecanismo utilizado
Tela recriada	Bundle, restauração de campos e valores selecionados.
Atividade editada	ActivityResultLauncher e identificação por UUID.
Dados salvos	SQLite, preferências ou arquivo privado, conforme a funcionalidade.
Consulta HTTP	ExecutorService ou thread, timeout, try/catch e fechamento da conexão.
Resposta atrasada	Contadores de geração e conferência do ciclo de vida.
Imagem em cartão reciclado	Marca de vínculo, referências fracas e limpeza no adaptador.
Falha de entrada	Erro no campo ou Toast e bloqueio do avanço.

Evitar alterar a tela errada

Quando o usuário troca a busca ou sai da Activity, uma resposta anterior pode chegar depois. Os controladores comparam um número de geração e o estado da Activity antes de aplicar dados. Esse padrão reduz resultados trocados e atualizações após fechar a tela.

Evitar perder dados

O editor mantém alterações temporárias até salvar. O banco atualiza pelo ID existente. A migração conserva as linhas anteriores. O arquivo de reservas usa escrita atômica. Cada mecanismo atende a um tipo de estado e deve continuar sendo usado pelos fluxos correspondentes.

Quando uma consulta falha, o app apresenta mensagem, conteúdo local ou imagem padrão. A precisão e a disponibilidade dos dados externos continuam ligadas ao fornecedor.

20 / VERIFICAÇÃO E APRESENTAÇÃO

Como testar e explicar o projeto

Fluxo manual principal

- Planeje uma viagem de três dias e avance pelo assistente.
- Adicione atividades em dois dias, deslize e toque nas abas.
- Edite, reordene, exclua e cancele uma exclusão.
- Gire a tela e confira dia e posição de rolagem.
- Salve, feche e reabra a viagem em Meus roteiros.
- Teste aeroportos sem rede e anote uma reserva.
- Com rede e configuração válidas, consulte mapa, câmbio, países e sugestões.

Testes existentes

PlanningUtilsTest cobre valores e validações. ItineraryUtilsTest cobre dias e ordenação. FlightSearchLinksTest verifica URLs. Os testes instrumentados incluem persistência, navegação, mapas, parsers, sugestões e ItineraryDayPagerTest para deslize, abas e restauração.

Últimos resultados registrados

Na implementação do pager em 09/11/2026, o build debug e nove testes unitários passaram. O teste instrumentado do pager passou após usar toque nativo. Na suíte completa, o teste existente de Explorar encontrou uma incompatibilidade do Espresso com a API 37. Esta preparação documental não repete os testes.

Comandos de referência

```
.\gradlew.bat assembleDebug testDebugUnitTest
.\gradlew.bat assembleDebugAndroidTest
.\gradlew.bat connectedDebugAndroidTest
```

Ao apresentar, conecte cada recurso a uma evidência do código: widgets no XML, ArrayList no roteiro, try/catch nos serviços, menu em BaseActivity e persistência em TripDatabase.

APÊNDICE / 1

Mapa dos arquivos Java

Os caminhos abaixo pertencem a app/src/main/java/com/example/malinha, exceto DebugMalinhaApplication, que fica em app/src/debug/java/com/example/malinha.

Arquivo	Função
activities/AiSuggestionsSheet.java	Cartões, seleção e novos pedidos de sugestões.
activities/BaseActivity.java	Toolbar, menu e insets compartilhados.
activities/CountryGuideActivity.java	Consultas de país e calendário.
activities/CurrencyActivity.java	Interface e execução da conversão de moedas.
activities/EditActivityActivity.java	Formulário e retorno da edição de uma atividade.
activities/ExploreActivity.java	Filtros, seções, favoritos e detalhes dos destinos.
activities/ExplorePlacesActivity.java	Busca e paginação de atrações e restaurantes.
activities/FeatureActivity.java	Estrutura remanescente das primeiras etapas.
activities/FlightPricesActivity.java	Tela legada de preços indicativos via proxy.
activities/FlightReservations.java	Criação, edição e exclusão das reservas anotadas.
activities/FlightsActivity.java	Busca externa de voos, aeroportos e histórico.
activities/HomeActivity.java	Atalhos, navegação e próximas viagens.
activities/HotelResultsActivity.java	Resultados de hospedagens pelo Places.

Consulte o capítulo do assunto antes de seguir os métodos da classe. Os arquivos legados foram identificados para distinguir o caminho atual de execução.

APÊNDICE / 2

Mapa dos arquivos Java

Os caminhos abaixo pertencem a app/src/main/java/com/example/malinha, exceto DebugMalinhaApplication, que fica em app/src/debug/java/com/example/malinha.

Arquivo	Função
activities/HotelsActivity.java	Especialização da pesquisa de hospedagem.
activities/ItineraryDraftActivity.java	Sessão de roteiro, dias, atividades e salvamento.
activities/MapActivity.java	Geocodificação, escolha de endereço e mapa.
activities/PlanTripActivity.java	Assistente e validação do planejamento.
activities/SplashActivity.java	Abertura e encaminhamento ao início.
activities/TravelSearchActivity.java	Formulário compartilhado mantido no fluxo de hotéis.
activities/TripsActivity.java	Lista, abertura, edição e exclusão de viagens.
adapters/ActivityAdapter.java	Cartões e ações de atividades.
adapters/CitySuggestionAdapter.java	Sugestões de cidades do Places.
adapters/CountryCalendarAdapter.java	Itens e seções do calendário de feriados.
adapters/DestinationAdapter.java	Cartões horizontais de destinos e favoritos.
adapters/ExplorePlaceAdapter.java	Cartões de locais com dados e ações Google.
adapters/HotelAdapter.java	Cartões de hospedagens.

Consulte o capítulo do assunto antes de seguir os métodos da classe. Os arquivos legados foram identificados para distinguir o caminho atual de execução.

APÊNDICE / 3

Mapa dos arquivos Java

Os caminhos abaixo pertencem a app/src/main/java/com/example/malinha, exceto DebugMalinhaApplication, que fica em app/src/debug/java/com/example/malinha.

Arquivo	Função
adapters/ItineraryDayAdapter.java	Páginas do roteiro e rolagem independente.
adapters/TripAdapter.java	Cartões e ações de viagens salvas.
database/TripDatabase.java	Esquema SQLite, migração e operações de persistência.
raiz/FirebaseAIService.java	Cliente assíncrono Gemini e mensagens de erro.
raiz/FirstFragment.java	Fragment inicial do template.
raiz/MainActivity.java	Activity do template preservado.
models/Atividade.java	Identidade, dia, horários, categoria, texto e custo.
models/CountryTravelData.java	Dados do país e estruturas do calendário.
models/Destino.java	IDs de recursos e região de um destino local.
models/SelectedCity.java	Identidade e coordenadas de cidade selecionada.
models/Viagem.java	Entidade de viagem e conjunto ordenado de atividades.
raiz/SecondFragment.java	Fragment secundário do template.
services/CitySearchRepository.java	Previsões de cidade, sessão e fetchPlace.

Consulte o capítulo do assunto antes de seguir os métodos da classe. Os arquivos legados foram identificados para distinguir o caminho atual de execução.

APÊNDICE / 4

Mapa dos arquivos Java

Os caminhos abaixo pertencem a app/src/main/java/com/example/malinha, exceto DebugMalinhaApplication, que fica em app/src/debug/java/com/example/malinha.

Arquivo	Função
services/CountryHttp.java	HTTP, códigos de país e limites das respostas.
services/CurrencyService.java	HTTP e cálculo de câmbio pela AwesomeAPI.
services/ExplorePlacesService.java	Consultas Places de atrações e restaurantes.
services/FlightPriceService.java	Consulta e validação de preços indicativos legados.
services/FlightSearchLinks.java	Configuração e geração dos links dos comparadores.
services/HotelPlacesService.java	Consulta Places voltada a hospedagens.
services/NagerDateService.java	Consulta e parse de feriados por país e ano.
services/PlacesClientProvider.java	Inicialização e obtenção do cliente Places.
services/RestCountriesService.java	Consulta autenticada e parse dos dados do país.
services/TravelSearchService.java	URLs externas, datas e abertura de navegador.
services/WikimediaImageService.java	Busca, validação e cache de metadados de fotos.
utils/AirportAutocomplete.java	Leitura local, filtro, ranking e rótulos de aeroportos.
utils/CityAutocompleteController.java	Debounce, estado e seleção no autocomplete.

Consulte o capítulo do assunto antes de seguir os métodos da classe. Os arquivos legados foram identificados para distinguir o caminho atual de execução.

APÊNDICE / 5

Mapa dos arquivos Java

Os caminhos abaixo pertencem a app/src/main/java/com/example/malinha, exceto DebugMalinhaApplication, que fica em app/src/debug/java/com/example/malinha.

Arquivo	Função
utils/CommonsPhotoView.java	Glide, placeholder, créditos e reciclagem de imagens.
utils/DestinationCatalog.java	24 destinos, categorias, países e destaques.
utils/HolidayWeekends.java	Cálculo local de intervalos prolongados.
utils/ItineraryUtils.java	Dias da viagem, horários, filtro e ordenação.
utils/PlaceDirections.java	Ações auxiliares para abrir direções.
utils/PlanningUtils.java	Datas e interpretação de valores monetários.
utils/ReadyItineraries.java	Criação de cópias dos oito roteiros locais.
raiz/DebugMalinhaApplication.java	Inicialização Firebase e App Check debug.

Consulte o capítulo do assunto antes de seguir os métodos da classe. Os arquivos legados foram identificados para distinguir o caminho atual de execução.

APÊNDICE / MANUTENÇÃO

Onde alterar cada recurso

Alteração desejada	Ponto de partida
Acrescentar destino	DestinationCatalog e textos em res/values.
Mudar roteiro pronto	ReadyItineraries, mantendo índices de dia válidos.
Mudar campos da atividade	Atividade, EditActivityActivity, ActivityAdapter e persistência.
Mudar dados da viagem	Viagem, PlanTripActivity e migração de TripDatabase.
Mudar o modelo Gemini	FirebaseAIService.MODEL e configuração do projeto Firebase.
Mudar aparência das sugestões	sheet_ai_suggestions.xml e item_ai_suggestion.xml.
Mudar links dos comparadores	FlightSearchLinks e seus testes unitários.
Atualizar aeroportos	assets/airports.tsv, atribuição e AirportAutocomplete.
Mudar as páginas de dias	ItineraryDraftActivity, ItineraryDayAdapter e layouts do pager.
Mudar fotos e cache	WikimediaImageService e CommonsPhotoView.
Mudar as cores	res/values/colors.xml e temas relacionados.

Limites atuais relevantes

As reservas de voo são anotações separadas. Associação com roteiro, checklist e lembretes continuam pendentes. A IA exige debug autorizado. O template inicial e o fluxo legado Travelpayouts permanecem no código. Os sites externos podem mudar seus parâmetros e a interpretação de filtros.

> Antes de mudar um campo persistido, defina como os dados já salvos serão lidos e migrados.

APÊNDICE / REFERÊNCIA

Glossário e fontes

Termo	Significado no projeto
Activity	Tela Android com ciclo de vida.
Intent	Pedido de navegação e transporte de dados.
Bundle	Estado temporário restaurável da interface.
Adapter / ViewHolder	Ligação entre dados e views reutilizáveis.
Callback / Future	Forma de receber o resultado de uma operação assíncrona.
Debounce	Espera curta após a última tecla antes de pesquisar.
IATA	Código de três letras usado para identificar aeroportos.
SQLite / JSON	Banco relacional e formato textual usado nas listas.
App Check	Verificação de autorização do ambiente que chama o Firebase.

Fontes para estudar

O texto foi preparado a partir das classes Java, layouts XML, manifesto, Gradle e README do projeto em C:/Users/maria/AndroidStudioProjects/Malinha. Os trechos de código foram extraídos dessa cópia.

Android Views e Activities: developer.android.com
Firebase AI Logic: firebase.google.com/docs/ai-logic
Maps e Places: developers.google.com/maps
AwesomeAPI: docs.awesomeapi.com.br
MediaWiki: mediawiki.org/wiki/Extension:PageImages
Commons: commons.wikimedia.org
Feriados: nagerholidays.com/api
Países: restcountries.com/docs/countries
Aeroportos: openflights.org/data.php

As fontes externas são referências de estudo. Versões, endpoints e valores de configuração descritos são os observados no código, sem exposição de chaves, tokens ou reservas pessoais.